Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea practică Nr. 5

Disciplina: Tehnici de programare

Tema: „Funcții, metode și importuri pe web (Python)”

A efectuat : st. gr.SI – 263 Alexandr Berejnoi

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

1. Importarea modulelor din biblioteca standard și externe (import, from … import, alias) și efectuarea cererilor web de bază – exercițiile 1–8;
2. Utilizarea metodelor și atributelor obiectelor web: răspuns HTTP, antete, șiruri de caractere, cookie-uri, sesiuni – exercițiile 9–20;
3. Scrierea funcțiilor proprii: parametri, valori implicite, docstring-uri, adnotări de tip, tratarea erorilor – exercițiile 21–34;
4. Utilizarea altor module din biblioteca standard (urllib.parse, re, html.parser, hashlib, json, socket, ssl) – exercițiile 35–43;
5. Crearea modulului propriu webtools.py, a programului main.py și a raportului site-ului în formate CSV și JSON – exercițiile 44–50.

Pregătirea mediului de lucru

Cerință: Verificați versiunea Python (python --version, 3.9 sau mai nouă), instalați biblioteca requests (pip install requests) și creați directorul web_lab, în care veți salva fișierele part1_imports.py, part2_methods.py etc., fiecare începând cu șablonul din lucrare (BASE_URL, ECHO_URL, TIMEOUT).

Lucrarea a fost realizată într-un mediu virtual Python (`.venv`), activ în terminal (prefixul `(.venv)` din prompt). Versiunea interpretatorului a fost verificată cu comanda `python --version`: rezultatul este **Python 3.14.2**, deci este îndeplinită cerința minimă (3.9 sau mai nouă).

Biblioteca externă `requests` a fost instalată cu comanda `pip install requests`. Pip a raportat mesajul „Requirement already satisfied” pentru `requests` (versiunea **2.34.2**) și pentru dependențele ei (`charset_normalizer`, `idna`, `urllib3`, `certifi`), adică biblioteca era deja prezentă în mediul virtual.

A fost creat directorul `web_lab`, în care au fost salvate fișierele lucrării: `part1_imports.py`, `part2_methods.py`, `part3_functions.py`, `part4_modules.py`, `webtools.py` și `main.py`. Fiecare fișier începe cu șablonul din lucrare:

```
# Laborator: functii, metode si importuri pe web
# Student: Berejnoi Alexandr

BASE_URL = "https://cybercor.org"
```

```
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde
```

Regulile de lucru cu un site real au fost respectate în toate fișierele: fiecare cerere are `timeout=TIMEOUT`, între cererile succesive există `time.sleep(1)`, către `cybercor.org` se trimit doar cereri GET și HEAD, nu există bucle infinite care trimit cereri, iar accesările se limitează la adresele indicate în exerciții. Codul sursă este publicat în repository-ul GitHub `AlexBerej/tehnici-de-programare`, directorul `L5/web_lab`.

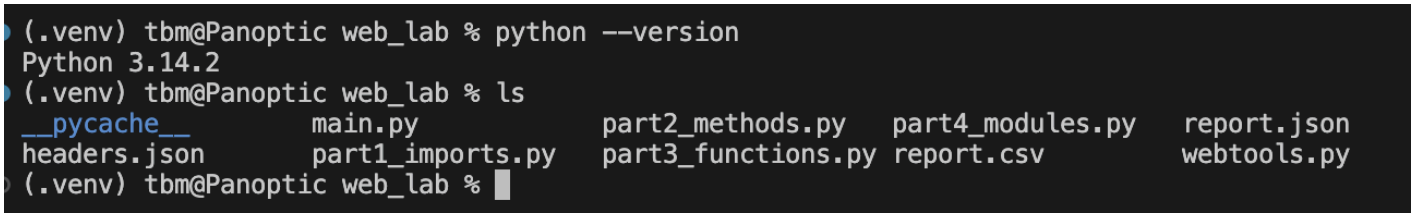


Figura 0.1 - Verificarea versiunii Python și a conținutului directorului web_lab

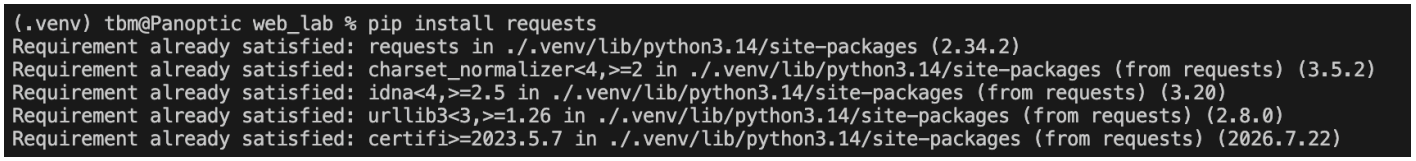


Figura 0.2 - Instalarea bibliotecii requests (pip install requests)

Partea 1. Importuri

Fișier de lucru: part1_imports.py (exercițiile 1–8).

Exercițiul 1. Primul import

Cerință: Importați requests și afișați versiunea acestuia (requests.__version__); apoi importați urllib.request. Explicați într-un comentariu de ce requests a necesitat pip install, iar urllib nu.

Au fost importate modulul extern `requests` și modulul din biblioteca standard `urllib.request`. Versiunea bibliotecii `requests` se afișează cu atributul `requests.__version__`.

```
# Exercitiu 1
import requests
print(requests.__version__)

import urllib.request

# requests este un modul extern, care trebuie instalat separat cu pip install requests
# iar urllib este un modul din biblioteca standard, care vine instalat odata cu Python
```

Rezultat: la rulare se afișează versiunea instalată a bibliotecii `requests` (2.34.2 în mediul de lucru). Importul `urllib.request` nu produce nicio ieșire.

`requests` nu face parte din biblioteca standard Python: este un pachet creat de terți, care se descarcă din depozitul PyPI și se instalează separat cu `pip install requests`. Modulul `urllib` este inclus în biblioteca standard și vine instalat odată cu interpretatorul Python, de aceea poate fi importat imediat, fără instalare.



Figura 1.1 - Exercițiul 1: importul modulelor requests și urllib.request, versiunea requests

Exercițiul 2. Două stiluri de import

Cerință: Descărcați pagina BASE_URL cu requests.get() și afișați codul de stare; rescrieți scriptul cu from requests import get. Scrieți în comentarii câte un avantaj al fiecărui stil.

Pagina `BASE_URL` a fost descărcată cu `requests.get()`, iar codul de stare a fost afișat cu `.status_code`. Apoi același script a fost rescris cu `from requests import get`. Ambele variante apelează aceeași funcție și afișează același rezultat: codul de stare al paginii principale (200 pentru o cerere reușită). Fiecare cerere are `timeout=TIMEOUT`, iar între cereri există `time.sleep(1)`.

```
# Exercitiu 2
# Avantajul stilului "import requests": se vede clar de unde vine functia (requests.get)
# si numele get nu intra in conflict cu alte nume din program
print(requests.get(BASE_URL, timeout=TIMEOUT).status_code)

time.sleep(1)

# Avantajul stilului "from requests import get": codul este mai scurt, scriem doar get
from requests import get
print(get(BASE_URL, timeout=TIMEOUT).status_code)
```

Avantajul stilului `import requests`: din cod se vede imediat de unde provine funcția (`requests.get`), iar numele `get` nu intră în conflict cu alte nume din program. **Avantajul stilului `from requests import get`:** codul este mai scurt, se scrie doar `get(...)`.

```
# Exercitiu 2
# Avantajul stilului "import requests": se vede clar de unde vine functia (requests.get)
# si numele get nu intra in conflict cu alte nume din program
print(requests.get(BASE_URL, timeout=TIMEOUT).status_code)

time.sleep(1)

# Avantajul stilului "from requests import get": codul este mai scurt, scriem doar get
from requests import get
print(get(BASE_URL, timeout=TIMEOUT).status_code)
```

Figura 1.2 - Exercițiul 2: stilurile import requests și from requests import get

Exercițiul 3. Alias-uri

Cerință: Importați requests cu alias (import requests as rq) și descărcați pagina principală cu rq.get(). Explicați într-un comentariu când un alias face codul mai ușor și când mai greu de citit.

Biblioteca a fost importată cu alias, `import requests as rq`, iar pagina principală a fost descărcată cu `rq.get()`. Se afișează același cod de stare ca în exercițiul 2.

```
# Exercitiu 3
import requests as rq
# Un alias face codul mai usor de citit cand modulul are un nume lung sau cand alias-ul
# este unul cunoscut de toata lumea (np pentru numpy, pd pentru pandas)
# Il face mai greu de citit cand este un alias criptic sau nestandard (rq pentru requests),
# pentru ca cititorul trebuie sa ghiceasca la ce se refera
print(rq.get(BASE_URL, timeout=TIMEOUT).status_code)
```

Un alias face codul mai ușor de citit când numele modulului este lung sau când alias-ul este unul cunoscut de toată lumea (`np` pentru numpy, `pd` pentru pandas). Îl face mai greu de citit când este criptic sau nestandard (`rq` pentru requests), pentru că cititorul trebuie să ghicească la ce modul se referă.

```python
# Exercitiu 3
import requests as rq
# Un alias face codul mai usor de citit cand modulul are un nume lung sau cand alias-ul
# este unul cunoscut de toata lumea (np pentru numpy, pd pentru pandas)
# Il face mai greu de citit cand este un alias criptic sau nestandard (rq pentru requests),
# pentru ca cititorul trebuie sa ghiceasca la ce se refera
print(rq.get(BASE_URL, timeout=TIMEOUT).status_code)
```

Figura 1.3 - Exercițiul 3: importul cu alias (import requests as rq)

Exercițiul 4. Doar biblioteca standard

Cerință: Descărcați pagina principală folosind doar urllib.request.urlopen(); afișați atributul .status și primele 200 de caractere din corp (octeții se decodifică în șir de caractere).

Pagina principală a fost descărcată folosind doar biblioteca standard, cu `urllib.request.urlopen()`. Obiectul returnat are atributul `.status` (codul de stare). Metoda `.read()` returnează octeți (`bytes`), care sunt transformați în șir de caractere cu `.decode("utf-8")`; din șir se afișează primele 200 de caractere prin feliere (`[:200]`).

```python
# Exercitiu 4
val = urllib.request.urlopen(BASE_URL, timeout=TIMEOUT)
print(val.status)
# .read() returneaza bytes, deci decodam raspunsul in UTF-8 pentru a-l afisa ca text
print(val.read().decode("utf-8")[:200])
```

Rezultat: se afișează codul de stare și începutul codului HTML al paginii (primele 200 de caractere).

```python
# Exercitiu 4
val = urllib.request.urlopen(BASE_URL, timeout=TIMEOUT)
print(val.status)
# .read() returneaza bytes, deci decodam raspunsul in UTF-8 pentru a-l afisa ca text
print(val.read().decode("utf-8")[:200])
```

Figura 1.4 - Exercițiul 4: descărcarea paginii cu urllib.request.urlopen()

Exercițiul 5. Priviți în interiorul unui modul

Cerință: Rulați print(dir(requests)); alegeți trei nume din rezultat și indicați într-un comentariu dacă fiecare este funcție, clasă sau modul.

Funcția `dir()` returnează lista numelor definite în modulul `requests`. Din rezultat au fost alese trei nume, iar tipul fiecăruia este indicat în comentarii.

```
# Exercitiu 5
print(dir(requests))
# get — functie
# Session — clasa
# adapters — modul
```

`get` este o **funcție** (trimite o cerere GET); `Session` este o **clasă** (obiect care păstrează setările și cookie-urile între cereri); `adapters` este un **modul** (submodul al pachetului `requests`, care conține adaptoarele de transport). Tipul se poate verifica și cu `type(requests.get)`, `type(requests.Session)` și `type(requests.adapters)`.

```
['ConnectTimeout', 'ConnectionError', 'DependencyWarning', 'FileModeWarning', 'HTTPError', 'JSONDecodeError', 'NullHandler', 'PreparedRequest', 'ReadT
imeout', 'Request', 'RequestException', 'RequestsDependencyWarning', 'Response', 'Session', 'Timeout', 'TooManyRedirects', 'URLRequired', '__all__', '
__author__', '__author_email__', '__build__', '__builtins__', '__cached__', '__cake__', '__copyright__', '__description__', '__doc__', '__file__', '__
license__', '__loader__', '__name__', '__package__', '__path__', '__spec__', '__title__', '__url__', '__version__', '_check_cryptography', '_internal_
utils', '_types', 'adapters', 'annotations', 'api', 'auth', 'certs', 'chardet_version', 'charset_normalizer_version', 'check_compatibility', 'codes',
'compat', 'cookies', 'delete', 'exceptions', 'get', 'head', 'hooks', 'logging', 'models', 'options', 'packages', 'patch', 'post', 'put', 'request', 's
ession', 'sessions', 'ssl', 'status_codes', 'structures', 'urllib3', 'utils', 'warnings']
```

Figura 1.5 - Exercițiul 5: conținutul modulului requests (dir(requests))

Exercițiul 6. Citiți documentația

Cerință: Rulați help(requests.get), găsiți parametrul care setează timpul maxim de așteptare și folosiți-l într-o cerere.

Comanda `help(requests.get)` afișează documentația funcției. Semnătura este `get(url, params=None, **kwargs)`: parametrii suplimentari se transmit prin `**kwargs`, descriși ca „argumente opționale pe care le acceptă `request`". Documentația funcției de bază `requests.request` (`help(requests.request)`) descrie parametrul **`timeout`**: numărul de secunde de așteptare a răspunsului serverului înainte de renunțare (număr sau tuplu `(connect timeout, read timeout)`). Parametrul a fost folosit într-o cerere reală, `get(BASE_URL, timeout=TIMEOUT)`.

```
# Exercitiu 6
help(requests.get)
# parametrul care seteaza timpul maxim de asteptare este timeout (in secunde)
time.sleep(1)
print(get(BASE_URL, timeout=TIMEOUT).status_code)
```

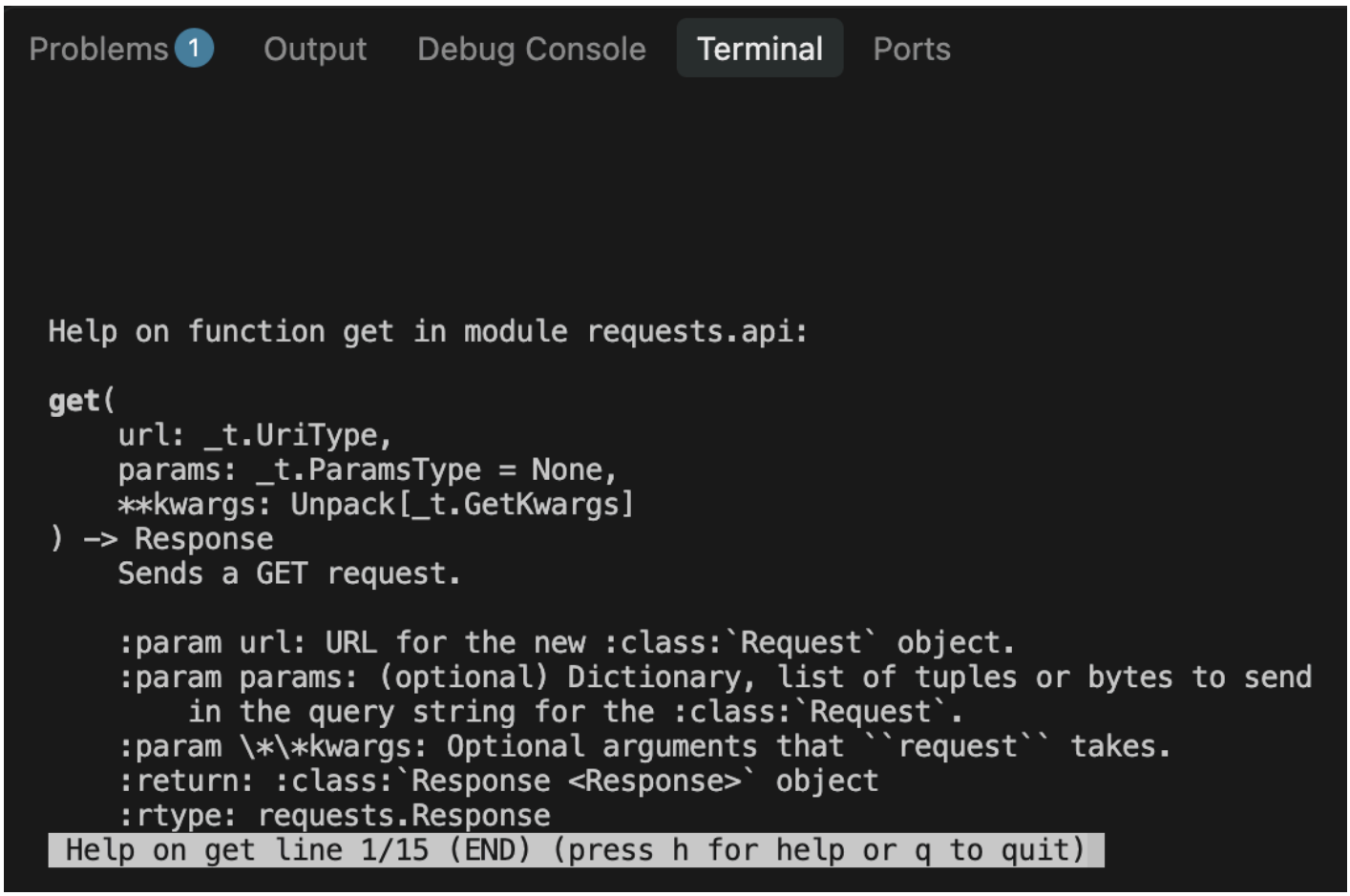


Figura 1.6 - Exercițiul 6: documentația funcției requests.get() și parametrul timeout

Exercițiul 7. Cronometrarea unei cereri

Cerință: Importați time și măsurați durata unei cereri către pagina principală cu time.perf_counter() (înainte și după); comparați rezultatul cu response.elapsed.

Modulul `time` a fost importat, iar durata cererii către pagina principală a fost măsurată cu `time.perf_counter()`, apelat înainte și după cerere; diferența `end - start` este durata în secunde. Rezultatul a fost comparat cu `response.elapsed`.

```
# Exercitiu 7
start = time.perf_counter()
response = get(BASE_URL, timeout=TIMEOUT)
end = time.perf_counter()

print(response.status_code)
print("Durata cu perf_counter:", end - start, "secunde")
print("Durata cu response.elapsed:", response.elapsed.total_seconds(), "secunde")
# perf_counter masoara tot timpul, inclusiv conectarea si citirea raspunsului,
# iar response.elapsed masoara doar intervalul dintre trimiterea cererii si primirea
# antetelor raspunsului, deci de obicei are o valoare putin mai mica
```

Rezultat: se afișează codul de stare și cele două durări, în secunde. Valorile sunt apropiate, dar nu identice.

`perf_counter()` măsoară tot intervalul cât durează apelul `get(...)`, inclusiv conectarea și citirea răspunsului, iar `response.elapsed` măsoară doar intervalul dintre trimiterea cererii și primirea antetelor răspunsului. De aceea `response.elapsed` are, de obicei, o valoare puțin mai mică.

```
Durata cu perf_counter: 0.31493445800151676 secunde
Durata cu response.elapsed: 0.3115 secunde
```

Figura 1.7 - Exercițiul 7: măsurarea duratei cererii cu time.perf_counter() și response.elapsed

Exercițiul 8. Când un import eșuează

Cerință: Scrieți un script care încearcă import bs4 într-un bloc try / except ImportError; dacă modulul lipsește, afișați mesajul de instalare cerut în lucrare.

Importul modulului `bs4` (BeautifulSoup) a fost încercat într-un bloc `try / except ImportError`. Dacă modulul este instalat, se afișează „bs4 este instalat”. Dacă lipsește, Python ridică `ImportError`, excepția este prinsă și se afișează mesajul cerut: „Instalați modulul cu: pip install beautifulsoup4”. În ambele cazuri programul continuă, fără oprire cu eroare.

```
# Exercitiu 8
try:
    import bs4
    print("bs4 este instalat")
except ImportError:
    print("Instalați modulul cu: pip install beautifulsoup4")
```

```
Durata cu response.elapsed: 0.3115 secunde
Instalați modulul cu: pip install beautifulsoup4
(.venv) tbm@Panoptic web_lab %
```

Figura 1.8 - Exercițiul 8: tratarea ImportError la importul modulului bs4

Întrebare de verificare

Întrebare: Care este diferența dintre un modul, un pachet și o bibliotecă?

Un **modul** este un singur fișier Python (`.py`) care conține cod reutilizabil (funcții, clase, constante) și se aduce în program cu `import`, de exemplu `time` sau `webtools.py`. Un **pachet** este un director care grupează mai multe module (și, eventual, subpachete), recunoscut în

mod obișnuit după fișierul `__init__.py`; de exemplu `requests` sau `urllib` (`urllib.request` este un submodul al pachetului `urllib`). O **bibliotecă** este un termen general, nu o construcție a limbajului: o colecție de cod reutilizabil, formată dintr-un modul sau din mai multe pachete (biblioteca standard Python, biblioteca `requests`).

Partea 2. Metode și atribute

Fișier de lucru: part2_methods.py (exercițiile 9–20).

Exercițiul 9. Elementele de bază ale răspunsului

Cerință: Descărcați pagina principală și afișați response.status_code, response.ok, response.url și response.encoding; lângă fiecare linie comentați dacă este atribut sau metodă.

Pagina principală a fost descărcată cu `requests.get()`, iar din obiectul `response` au fost afișate patru valori. Toate patru sunt **atribute** (valori deja calculate, citite fără paranteze), nu metode.

```
# Exercitiu 9
response = requests.get(BASE_URL, timeout=TIMEOUT)
print(response.status_code)  # atribut
print(response.ok)           # atribut
print(response.url)          # atribut
print(response.encoding)     # atribut
```

Rezultat: se afișează codul de stare (`status_code`), valoarea booleană `ok` (True pentru coduri de stare sub 400), adresa URL a răspunsului și codificarea textului (`encoding`, de exemplu `utf-8`).

Diferența se vede în sintaxă: un atribut se citește fără paranteze (`response.ok`), iar o metodă este o acțiune apelată cu paranteze (`response.json()`, `response.raise_for_status()`).

```
(.venv) tbm@Panoptic web_lab % python3 part2_methods.py
200
True
https://cybercor.org/
utf-8
```

Figura 2.1 - Exercițiul 9: atributele de bază ale obiectului response

Exercițiul 10. Tratarea erorilor cu o metodă

Cerință: Apelați response.raise_for_status() pe pagina principală; apoi trimiteți o cerere către BASE_URL + "/this-page-does-not-exist", prindeți requests.HTTPError și afișați un mesaj prietenos în loc de oprirea programului.

Metoda `response.raise_for_status()` a fost apelată pe pagina principală: pentru un cod de stare reușit nu face nimic și programul afișează „Pagina principala este in regula”. Apoi a fost trimisă (după o pauză de o secundă) o cerere către `BASE_URL + "/this-page-does-not-exist"`. Pentru coduri de eroare 4xx/5xx metoda ridică excepția `requests.HTTPError`; aceasta este prinsă cu `try / except`, iar programul afișează un mesaj prietenos (împreună cu textul erorii) în loc să se oprească.

```
# Exercitiu 10
response.raise_for_status()  # metoda: nu face nimic daca cererea s-a terminat bine
print("Pagina principala este in regula")

time.sleep(1)

try:
    bad = requests.get(BASE_URL + "/this-page-does-not-exist", timeout=TIMEOUT)
    bad.raise_for_status()  # pentru 404 ridica requests.HTTPError
except requests.HTTPError as e:
    print("Pagina ceruta nu exista:", e)
```



Figura 2.2 - Exercițiul 10: tratarea erorii HTTP cu raise_for_status()

Exercițiul 11. Toate antetele

Cerință: Parcurgeți response.headers.items() și afișați fiecare antet în forma Nume: valoare.

Antetele răspunsului se află în `response.headers`, un dicționar special. Metoda `.items()` returnează perechile (nume, valoare), care sunt parcurse cu un ciclu `for` și afișate în forma `Nume: valoare`.

```
# Exercitiu 11
for name, value in response.headers.items():
    print(f"{name}: {value}")
```

Rezultat: se afișează lista completă a antetelor primite de la server (de exemplu `Server`, `Content-Type`, `Date`), câte unul pe linie.

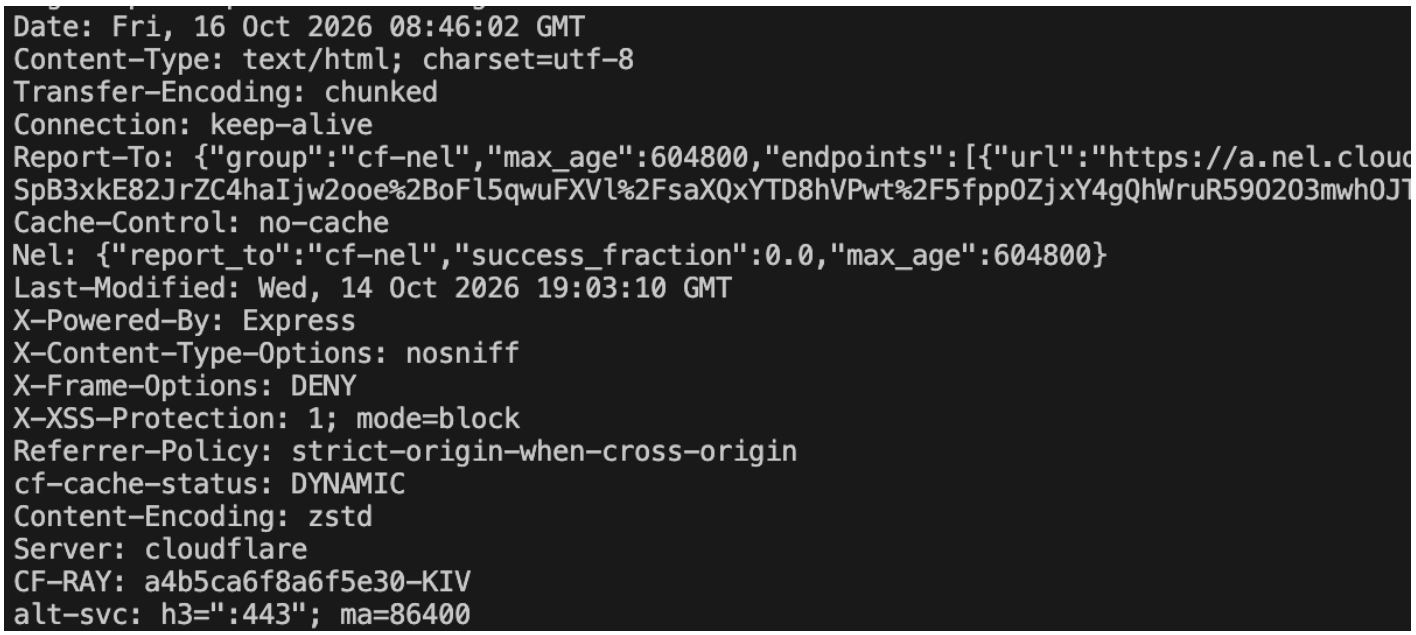


Figura 2.3 - Exercițiul 11: antetele răspunsului (response.headers.items())

Exercițiul 12. Metode de dicționar

Cerință: Cu response.headers.get() afișați antetele Server și Content-Type (valoare implicită „lipsește”); încercați și varianta „content-type”, cu litere mici, și notați ce observați.

Metoda `response.headers.get(cheie, implicit)` returnează valoarea antetului sau valoarea implicită („lipsește”) dacă antetul nu există, fără să ridice `KeyError`. Au fost afișate antetele `Server` și `Content-Type`, apoi încă o dată `Content-Type`, scris cu litere mici.

```
# Exercitiu 12
print(response.headers.get("Server", "lipsește"))
print(response.headers.get("Content-Type", "lipsește"))
print(response.headers.get("content-type", "lipsește"))
# Cu litere mici se obtine exact aceeasi valoare, pentru ca response.headers
# este un dictionar care ignora diferenta dintre litere mari si mici
```

Observație: căutarea cu `"content-type"` (litere mici) returnează exact aceeași valoare ca `"Content-Type"`.

`response.headers` este un `CaseInsensitiveDict`, adică un dicționar care ignoră diferența dintre litere mari și mici în numele cheilor, deoarece standardul HTTP definește numele antetelor ca insensibile la majuscule. Un `dict` obișnuit ar fi tratat cele două scrieri ca fiind chei diferite.

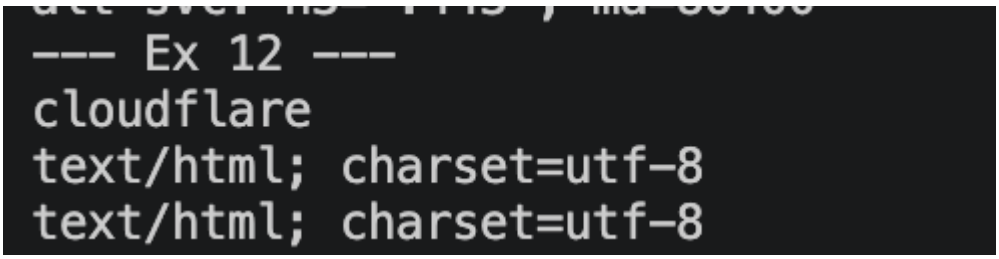


Figura 2.4 - Exercițiul 12: citirea antetelor cu headers.get()

Exercițiul 13. Înlănțuirea metodelor de șir

Cerință: Numărați de câte ori apare „cyber” pe pagina principală cu response.text.lower().count("cyber"); explicați într-un comentariu de ce se pot înlănțui .lower() și .count().

Cuvântul „cyber” a fost numărat pe pagina principală cu `response.text.lower().count("cyber")`. Textul este mai întâi transformat în litere mici, astfel încât să fie numărate toate variantele (`Cyber`, `CYBER`, `cyber`).

```
# Exercitiu 13
cyber_count = response.text.lower().count("cyber")
print("Cuvantul cyber apare de", cyber_count, "ori")
# Putem inlantui metodele pentru ca response.text este un sir (str), .lower() returneaza
# tot un sir, iar pe acest sir se poate apela imediat urmatoarea metoda count()
```

Rezultat: se afișează numărul de apariții (sunt numărate și aparițiile din codul HTML, de exemplu din adrese și atribute, nu doar din textul vizibil).

Metodele pot fi înlănțuite pentru că `response.text` este un șir (`str`), `.lower()` returnează tot un șir (unul nou), iar pe acest șir se poate apela imediat următoarea metodă, `.count()`. Expresia se evaluează de la stânga la dreapta: `response.text` → `.lower()` → `.count("cyber")`.



Figura 2.5 - Exercițiul 13: numărarea cuvântului „cyber” cu lower() și count()

Exercițiul 14. Găsirea titlului

Cerință: Extrageți textul dintre <title> și </title> folosind .find() și felierea (slicing) pe response.text; curățați-l cu .strip() și afișați-l.

Metoda `.find()` returnează indexul de început al lui `<title>`; la el se adaugă `len("<title>")`, pentru a obține poziția primului caracter al titlului. Poziția lui `</title>`

se obține tot cu `.find()`. Textul dintre cele două poziții se extrage prin feliere, `html[first:last]`, și se curăță cu `.strip()`. Căutarea se face în `html.lower()`, astfel încât să fie găsite și tag-urile scrise cu majuscule, iar felierea se aplică textului original. Dacă tag-ul lipsește (`find` returnează -1), se afișează un mesaj.

```
# Exercitiu 14
html = response.text
start = html.lower().find("<title>")
#start este indexul de unde incepe tagul
if start == -1:
    print("Pagina nu are tag <title>")
else:
    first = start + len("<title>")
    last = html.lower().find("</title>")
    #print(first)
    #print(last)
    title = html[first:last].strip()
    print("Titlu:", title)
```

Rezultat: se afișează titlul paginii, fără spațiile și liniile goale din jurul lui.



Figura 2.6 - Exercițiul 14: extragerea titlului paginii cu find() și feliere

Exercițiul 15. Numărarea liniilor

Cerință: Împărțiți codul HTML în linii cu .splitlines(); afișați numărul de linii și lungimea celei mai lungi linii (max(lines, key=len)).

Metoda `.splitlines()` împarte codul HTML într-o listă de linii. `len(lines)` este numărul de linii, iar `max(lines, key=len)` returnează cea mai lungă linie (cheia de comparație este lungimea), a cărei lungime se afișează cu `len()`.

```
# Exercitiu 15
lines = html.splitlines()
print("Numar de linii:", len(lines))
print("Lungimea celei mai lungi linii:", len(max(lines, key=len)))
```

Rezultat: se afișează numărul de linii ale paginii și lungimea celei mai lungi linii.

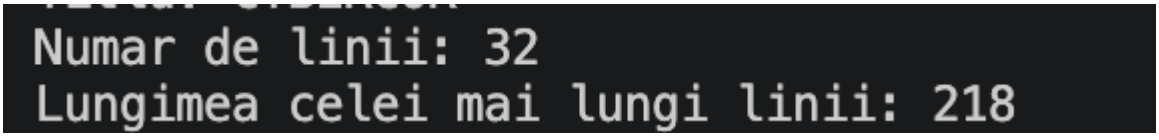


Figura 2.7 - Exercițiul 15: numărul de linii și lungimea celei mai lungi linii

Exercițiul 16. Verificarea HTTPS

Cerință: Folosiți .startswith() pentru a verifica dacă response.url începe cu https:// și afișați „Conexiune securizată” sau „Conexiune nesecurizată”.

Expresia `response.url.startswith("https://")` returnează `True` sau `False`, iar în funcție de rezultat se afișează „Conexiune securizată” sau „Conexiune nesecurizată”. Cererea a fost trimisă către o adresă `https://`, deci se afișează „Conexiune securizată”.

```
# Exercitiu 16
if response.url.startswith("https://"):
    print("Conexiune securizată")
else:
    print("Conexiune nesecurizată")
```



Figura 2.8 - Exercițiul 16: verificarea conexiunii HTTPS cu startswith()

Exercițiul 17. Urmărirea redirecționărilor

Cerință: Trimiteți o cerere către http://cybercor.org (fără S); parcurgeți response.history, afișând codul de stare și URL-ul fiecărei redirecționări, apoi URL-ul final.

Cererea a fost trimisă către `http://cybercor.org` (HTTP simplu, fără S). Biblioteca `requests` urmărește automat redirecționările, iar acestea sunt păstrate în lista `response.history` (câte un obiect `Response` pentru fiecare pas). Pentru fiecare pas se afișează codul de stare și URL-ul, apoi URL-ul final (`response.url`).

```
# Exercitiu 17
response = requests.get("http://cybercor.org", timeout=TIMEOUT)
for step in response.history:
    print(step.status_code, step.url)
print("URL final:", response.url)
```

Rezultat: dacă serverul redirecționează de la HTTP la HTTPS, lista conține un pas cu codul de stare 301 (sau 308), iar URL-ul final începe cu `https://`. Dacă nu există redirecționări, lista `history` este goală și se afișează doar URL-ul final.



Figura 2.9 - Exercițiul 17: lanțul de redirecționări (response.history)

Exercițiul 18. HEAD versus GET

Cerință: Trimiteți către pagina principală o cerere requests.head() și una requests.get(); comparați len(response.content) și explicați diferența într-un comentariu.

Către pagina principală au fost trimise o cerere `requests.head()` și o cerere `requests.get()`, cu pauză de o secundă între ele, iar lungimea corpului răspunsului, `len(response.content)`, a fost comparată.

```
# Exercitiu 18
head_response = requests.head(BASE_URL, timeout=TIMEOUT)
time.sleep(1)
get_response = requests.get(BASE_URL, timeout=TIMEOUT)
print("HEAD:", len(head_response.content), "octeti")
print("GET:", len(get_response.content), "octeti")
# HEAD cere doar antetele, serverul nu trimite corpul paginii, deci len(content) este 0
# GET cere si corpul, deci content contine tot codul HTML al paginii.
```

Rezultat: pentru HEAD se afișează 0 octeți, iar pentru GET numărul de octeți ai codului HTML al paginii.

Metoda HEAD cere doar antetele: serverul nu trimite corpul răspunsului, deci `content` este gol (0 octeți). Metoda GET cere și corpul, deci `content` conține tot codul HTML al paginii. HEAD este utilă pentru verificări rapide (existența paginii, tipul și dimensiunea declarate în antete), fără descărcarea conținutului.

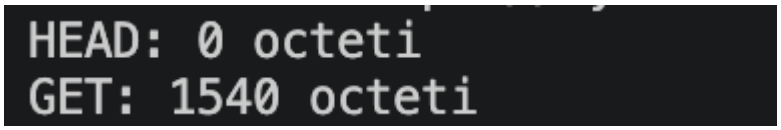


Figura 2.10 - Exercițiul 18: compararea cererilor HEAD și GET

Exercițiul 19. Cookie-uri

Cerință: Parcurgeți response.cookies și afișați pentru fiecare cookie atributele .name și .secure; dacă nu există cookie-uri, afișați „Niciun cookie setat”.

Cookie-urile răspunsului se găsesc în `response.cookies` și se parcurg cu `for`; pentru fiecare cookie se afișează atributele `.name` și `.secure`. Dacă serverul nu a setat cookie-uri, `response.cookies` este gol și se afișează „Niciun cookie setat”. În acest punct al scriptului variabila `response` conține răspunsul din exercițiul 17 (pagina finală, obținută după redirecționare), deci sunt verificate cookie-urile acelei cereri.

```
# Exercitiu 19
for cookie in response.cookies:
    print(cookie.name, cookie.secure)
if not response.cookies:
    print("Niciun cookie setat")
```



Figura 2.11 - Exercițiul 19: cookie-urile răspunsului

Exercițiul 20. Sesiuni

Cerință: Creați un obiect requests.Session(), setați un User-Agent personalizat (WebLab-<numele vostru>) cu session.headers.update(), trimiteți o cerere către ECHO_URL + "/headers" și confirmați în rezultat că antetul a fost trimis.

A fost creat un obiect `requests.Session()`, iar cu `session.headers.update()` a fost setat un `User-Agent` personalizat (`WebLab-Berejnoi-Alexandr`), care este trimis cu toate cererile sesiunii. Cererea a fost trimisă către `ECHO_URL + "/headers"` (serviciul httpbin.org, care returnează în format JSON antetele primite), iar din răspuns, cu `echo.json()["headers"]`, a fost extras dicționarul antetelor.

```
# Exercitiu 20
sesiune = requests.Session()
sesiune.headers.update({"User-Agent": "WebLab-Berejnoi-Alexandr"})
echo = sesiune.get(ECHO_URL + "/headers", timeout=TIMEOUT)
sent_headers = echo.json()["headers"]
print("Header: ", sent_headers)
print("User-Agent trimis:", sent_headers.get("User-Agent", "lipsește"))
```

Rezultat: în dicționarul afișat apare `User-Agent: WebLab-Berejnoi-Alexandr`, ceea ce confirmă că antetul personalizat a fost trimis.

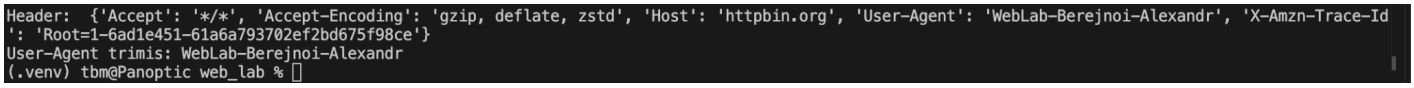


Figura 2.12 - Exercițiul 20: sesiune cu User-Agent personalizat (httpbin.org/headers)

Întrebare de verificare

Întrebare: Atât response.text, cât și response.json() oferă corpul răspunsului. De ce unul are paranteze, iar celălalt nu?

`response.text` este un **atribut**: o valoare deja pregătită (corpul răspunsului decodat ca șir), care se citește fără paranteze. `response.json()` este o **metodă**: o acțiune care analizează corpul răspunsului și construiește un dicționar sau o listă Python (și poate da eroare dacă răspunsul nu este JSON valid), deci trebuie apelată cu paranteze.

Partea 3. Funcții proprii

Fișier de lucru: part3_functions.py (exercițiile 21–34). Fiecare exercițiu este o funcție apelată cel puțin o dată; începând cu această parte, fiecare funcție trebuie să aibă docstring.

Exercițiul 21. Prima funcție

Cerință: Scrieți fetch(url), care returnează obiectul răspuns; apelați-o pentru BASE_URL și afișați codul de stare.

A fost scrisă funcția `fetch(url)`, care descarcă adresa și returnează întreg obiectul răspuns (mai jos este forma finală, cu parametrul implicit din exercițiul 23 și adnotările din exercițiul 26). Funcția a fost apelată pentru `BASE_URL`, iar codul de stare a fost afișat din obiectul returnat.

```
def fetch(url: str, timeout: int = TIMEOUT) -> requests.Response:
    """Descarca adresa url si returneaza obiectul raspuns (timeout implicit: 10 secunde)."""
    return requests.get(url, timeout=timeout)

# Exercitiu 21
response = fetch(BASE_URL)
print(response.status_code)
```

Rezultat: se afișează codul de stare al paginii principale.

```
(.venv) tbm@Panoptic web_lab % python3 part3_functions.py
200
```

Figura 3.1 - Exercițiul 21: funcția fetch(url)

Exercițiul 22. Returnarea unei singure valori

Cerință: Scrieți get_status(url), care returnează doar codul de stare; apelați-o pentru /, /robots.txt și /sitemap.xml, cu time.sleep(1) între apeluri.

Funcția `get_status(url)` apelează `fetch()` și returnează doar `.status_code` (un număr întreg). A fost apelată pentru căile `/`, `/robots.txt` și `/sitemap.xml`, cu `time.sleep(1)` între apeluri.

```
def get_status(url: str) -> int:
    """Returneaza codul de stare HTTP pentru adresa url."""
    return fetch(url).status_code

# Exercitiu 22
for path in ["/", "/robots.txt", "/sitemap.xml"]:
    print(path, get_status(BASE_URL + path))
```

Rezultat: pentru fiecare cale se afișează codul de stare (200 dacă resursa există, 404 dacă lipsește).



Figura 3.2 - Exercițiul 22: funcția get_status(url)

Exercițiul 23. Parametri impliciți

Cerință: Modificați funcția în fetch(url, timeout=10); apelați-o o dată cu valoarea implicită și o dată cu timeout=3.

Funcția `fetch` a primit parametrul `timeout` cu valoarea implicită `TIMEOUT` (10 secunde): `def fetch(url, timeout=TIMEOUT)`. Funcția a fost apelată o dată fără al doilea argument (se folosește valoarea implicită, 10 secunde) și o dată cu `timeout=3`.

```
# Exercitiu 23
print(fetch(BASE_URL).status_code)             # cu valoarea implicita (10 secunde)
time.sleep(1)
print(fetch(BASE_URL, timeout=3).status_code)  # cu timeout=3
```

Rezultat: în ambele cazuri se afișează codul de stare; diferă doar timpul maxim de așteptare al cererii.



Figura 3.3 - Exercițiul 23: parametrul implicit timeout al funcției fetch

Exercițiul 24. O singură sarcină pe funcție

Cerință: Scrieți get_title(html), care primește un șir HTML și returnează titlul paginii (logica din exercițiul 14), fără operații de rețea.

Funcția `get_title(html)` primește un șir HTML și returnează titlul paginii (logica din exercițiul 14, mutată într-o funcție); dacă nu există `<title>`, returnează `None`. Funcția nu face nicio operație de rețea: descărcarea (`fetch`) și analiza (`get_title`) sunt acum separate, deci `get_title` poate fi testată pe orice șir HTML, fără conexiune la internet.

```
def get_title(html: str) -> Optional[str]:
    """Returneaza titlul paginii (textul dintre <title> si </title>) dintr-un sir HTML.

    Returneaza None daca pagina nu are tag <title>. Functia nu face operatii de retea.
    """
    lower_html = html.lower()
    tag_start = lower_html.find("<title>")
```

```
    if tag_start == -1:
        return None
    start = tag_start + len("<title>")
    end = lower_html.find("</title>")
    return html[start:end].strip()

# Exercitiu 24
print(get_title(response.text))
```

Rezultat: se afișează titlul paginii principale.



Figura 3.4 - Exercițiul 24: funcția get_title(html)

Exercițiul 25. Docstring-uri

Cerință: Adăugați un docstring funcției get_title și afișați-l cu help(get_title).

Funcției `get_title` i s-a adăugat un docstring: un șir între ghilimele triple, plasat imediat după antetul funcției. Acesta descrie ce returnează funcția, cazul `None` și faptul că funcția nu folosește rețeaua. Docstring-ul este afișat cu `help(get_title)`.

```
# Exercitiu 25
help(get_title)
```

```
Help on function get_title in module __main__:

get_title(html: str) -> Optional[str]
    Returneaza titlul paginii (textul dintre <title> si </title>) dintr-un sir HTML.

    Returneaza None daca pagina nu are tag <title>. Functia nu face operatii de retea.
```

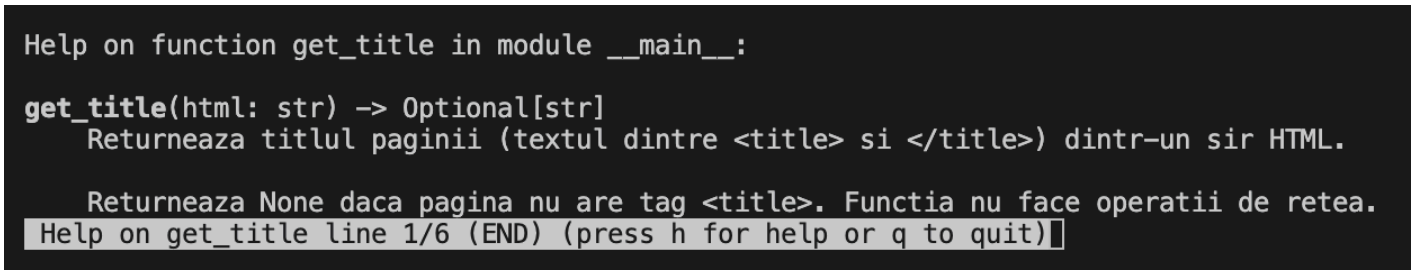


Figura 3.5 - Exercițiul 25: docstring-ul funcției get_title (help)

Exercițiul 26. Adnotări de tip

Cerință: Adăugați adnotări de tip funcțiilor (de ex. def get_status(url: str) -> int:); apelați get_status(123), observați ce se întâmplă și precizați dacă adnotările vă opresc.

Funcțiile au primit adnotări de tip, pentru parametri (`url: str`) și pentru valoarea returnată (`-> int`, `-> dict`, `-> Optional[str]`). Apoi a fost apelată `get_status(123)`, cu un număr întreg în loc de șir, într-un bloc `try / except requests.RequestException`.

```
def get_status(url: str) -> int:
    """Returneaza codul de stare HTTP pentru adresa url."""
    return fetch(url).status_code

# Exercitiu 26
try:
    print(get_status(123))
except requests.RequestException as e:
    print("Eroare:", type(e).__name__, "-", e)
# Adnotarile de tip nu ne opresc: Python nu le verifica la rulare, ele sunt doar
# documentatie pentru oameni si pentru unelte (de ex. mypy). Apelul get_status(123)
# a pornit normal, iar eroarea vine din requests, care nu stie sa foloseasca "123" ca URL
```

Rezultat: apelul nu este oprit de adnotare. Eroarea apare abia în interiorul bibliotecii `requests`, care ridică `MissingSchema` („Invalid URL '123': No scheme supplied”), o subclasă a lui `RequestException`. Excepția este prinsă și se afișează o linie de forma `Eroare: MissingSchema - ...`.

Adnotările de tip nu ne opresc. Python nu le verifică la rulare: ele sunt doar documentație pentru oameni și pentru unelte de analiză statică (de exemplu `mypy` sau editorul de cod). Apelul `get_status(123)` a pornit normal, iar eroarea vine din `requests`, care nu poate folosi `123` ca URL.

```
Eroare: MissingSchema - Invalid URL '123': No scheme supplied. Perhaps you meant https://123?
```

Figura 3.6 - Exercițiul 26: adnotări de tip și apelul get_status(123)

Exercițiul 27. Fără opriri neașteptate

Cerință: Scrieți page_exists(url), care returnează True sau False, folosind try / except requests.RequestException; verificați și pentru https://this-domain-does-not-exist.invalid.

Funcția `page_exists(url)` returnează `True` sau `False`. Cererea este făcută într-un bloc `try / except requests.RequestException`: orice eroare de rețea (domeniu inexistent, timeout, conexiune refuzată) este prinsă și transformată în `False`, în loc să oprească programul. Dacă cererea reușește, se returnează `response.ok`.

```
def page_exists(url: str) -> bool:
    """Returneaza True daca pagina raspunde fara eroare si False in caz contrar.

    Erorile de retea (domeniu inexistent, timeout etc.) nu opresc programul.
    """
    try:
        return fetch(url).ok
    except requests.RequestException:
        return False
```

```
# Exercitiu 27
print(page_exists(BASE_URL))
print(page_exists("https://this-domain-does-not-exist.invalid"))
```

Rezultat: pentru `BASE_URL` se afișează `True`, iar pentru `https://this-domain-does-not-exist.invalid` se afișează `False`: domeniul nu poate fi rezolvat (`ConnectionError`), dar programul continuă normal.

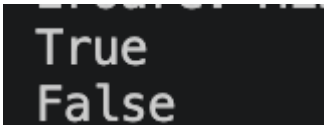


Figura 3.7 - Exercițiul 27: funcția page_exists(url)

Exercițiul 28. Returnarea unui dicționar

Cerință: Scrieți check_paths(base, paths), care primește o listă de căi și returnează un dicționar {path: status_code}.

Funcția `check_paths(base, paths)` parcurge lista de căi, apelează `get_status(base + path)` pentru fiecare și returnează un dicționar `{cale: cod_de_stare}`. Dacă o cerere eșuează, valoarea căii este `None`; între cereri se face o pauză de o secundă.

```
def check_paths(base: str, paths: list) -> dict:
    """Returneaza un dictionar {cale: cod_de_stare} pentru fiecare cale din lista paths.

    Daca o cerere esueaza, valoarea pentru acea cale este None. Intre cereri
    se face o pauza de o secunda.
    """
    results = {}
    for path in paths:
        try:
            results[path] = get_status(base + path)
        except requests.RequestException:
            results[path] = None
        time.sleep(1)
    return results

# Exercitiu 28
print(check_paths(BASE_URL, ["/", "/robots.txt", "/sitemap.xml"]))
```

Rezultat: se afișează un dicționar cu trei intrări, pentru `/`, `/robots.txt` și `/sitemap.xml`.



Figura 3.8 - Exercițiul 28: funcția check_paths(base, paths)

Exercițiul 29. Argumente cu nume

Cerință: Scrieți get_header(url, name, default="lipsește"), care returnează valoarea unui antet; apelați-o cu argumente cu nume: get_header(BASE_URL, name="Server").

Funcția `get_header(url, name, default="lipsește")` returnează valoarea unui singur antet, cu `headers.get(name, default)`. Apelul folosește argument cu nume, `name="Server"`; parametrul `default` nu este transmis, deci se folosește valoarea implicită.

```
def get_header(url: str, name: str, default: str = "lipsește") -> str:
    """Returneaza valoarea antetului name de la adresa url, sau default daca lipseste."""
    return fetch(url).headers.get(name, default)

# Exercitiu 29
print(get_header(BASE_URL, name="Server"))
```

Rezultat: se afișează valoarea antetului `Server` sau „lipsește”, dacă serverul nu îl trimite.

cloudflare

Figura 3.9 - Exercițiul 29: funcția get_header cu argumente cu nume

Exercițiul 30. Antete de securitate

Cerință: Scrieți security_headers(url), care verifică prezența celor cinci antete de securitate indicate în lucrare și returnează {antet: True/False}.

Funcția `security_headers(url)` descarcă pagina și verifică, printr-o comprehensiune de dicționar, dacă fiecare dintre cele cinci antete din lista `SECURITY_HEADERS` se găsește în `response.headers`. Returnează un dicționar `{antet: True/False}`. Operatorul `in` ignoră diferența dintre litere mari și mici, deoarece `headers` este un `CaseInsensitiveDict`.

```
SECURITY_HEADERS = [
    "Strict-Transport-Security",
    "Content-Security-Policy",
    "X-Frame-Options",
    "X-Content-Type-Options",
    "Referrer-Policy",
]

def security_headers(url: str) -> dict:
    """Verifica cele cinci antete de securitate si returneaza {antet: True/False}."""
    headers = fetch(url).headers
    return {name: name in headers for name in SECURITY_HEADERS}

# Exercitiu 30
results = security_headers(BASE_URL)
print(results)
```

Rezultat: se afișează dicționarul cu cele cinci antete și valoarea `True` sau `False` pentru fiecare.



Figura 3.10 - Exercițiul 30: funcția security_headers(url)

Exercițiul 31. Funcții care apelează funcții

Cerință: Scrieți score_headers(results), care primește dicționarul din exercițiul 30 și returnează un șir de tipul „3/5”; apelați score_headers(security_headers(BASE_URL)).

Funcția `score_headers(results)` primește dicționarul din exercițiul 30 și returnează un șir de forma „N/5”: `sum(results.values())` numără valorile `True` (care se comportă ca 1), iar `len(results)` este numărul total de antete. Cele două funcții au fost apelate împreună, `score_headers(security_headers(BASE_URL))`: rezultatul unei funcții este transmis direct ca argument celeilalte.

```
def score_headers(results: dict) -> str:
    """Primeste dictionarul din security_headers si returneaza scorul, de ex. '3/5'."""
    return f"{sum(results.values())}/{len(results)}"

# Exercitiu 31
print(score_headers(results))
print(score_headers(security_headers(BASE_URL)))
```

Rezultat: se afișează scorul antetelor de securitate, de forma `3/5` (numărul de antete prezente din cele cinci verificate).

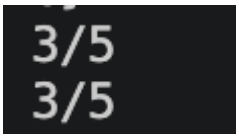


Figura 3.11 - Exercițiul 31: funcția score_headers(results)

Exercițiul 32. Două funcții, o sarcină

Cerință: Scrieți fetch_robots(base) (returnează textul /robots.txt sau None) și disallowed_paths(robots_text) (returnează lista valorilor Disallow:); a doua trebuie să funcționeze și pentru None.

Funcția `fetch_robots(base)` descarcă `/robots.txt` și returnează textul fișierului sau `None`, dacă fișierul lipsește (cod diferit de 200) ori cererea eșuează. Funcția `disallowed_paths(robots_text)` parcurge liniile, elimină comentariile (textul de după `#`), caută liniile care încep cu `Disallow:` și returnează lista valorilor nevide. Pentru `None` returnează lista goală, deci funcția poate fi apelată fără verificări suplimentare. Fiecare funcție are o singură sarcină, iar cele două se combină: `disallowed_paths(fetch_robots(BASE_URL))`.

```
def fetch_robots(base: str) -> Optional[str]:
    """Returneaza textul fisierului /robots.txt sau None daca fisierul lipseste."""
    try:
        response = fetch(base + "/robots.txt")
    except requests.RequestException:
        return None
    if response.status_code == 200:
        return response.text
    return None

def disallowed_paths(robots_text: Optional[str]) -> list:
    """Returneaza lista valorilor Disallow din textul robots.txt (lista goala pentru None)."""
    if robots_text is None:
        return []
    paths = []
    for line in robots_text.splitlines():
        line = line.split("#")[0].strip()  # eliminam comentariile
        if line.lower().startswith("disallow:"):
            value = line.split(":", 1)[1].strip()
            if value:  # "Disallow:" gol inseamna ca nu este interzis nimic
                paths.append(value)
    return paths

# Exercitiu 32
robots = fetch_robots(BASE_URL)
print(disallowed_paths(robots))
print(disallowed_paths(None))  # functioneaza si pentru None
```

Rezultat: primul apel afișează lista căilor interzise din `robots.txt`, iar al doilea (apelul cu `None`) afișează lista goală:

```
[]
```



Figura 3.12 - Exercițiul 32: funcțiile fetch_robots și disallowed_paths

Exercițiul 33. Oricâte argumente

Cerință: Scrieți response_times(*urls), care returnează un dicționar {url: secunde}; apelați-o cu două URL-uri.

Funcția `response_times(*urls)` primește oricâte adrese (`*urls` le colectează într-un tuplu), măsoară cu `time.perf_counter()` durata fiecărei cereri și returnează dicționarul `{url: secunde}` (`None` pentru cererile eșuate), cu pauză de o secundă între cereri. A fost apelată cu două URL-uri: `BASE_URL` și `BASE_URL + "/robots.txt"`.

```
def response_times(*urls: str) -> dict:
    """Returneaza un dictionar {url: secunde} cu durata cererii pentru fiecare url.

    Daca o cerere esueaza, valoarea este None. Intre cereri se face o pauza de o secunda.
    """
```

```
    times = {}
    for url in urls:
        start = time.perf_counter()
        try:
            fetch(url)
            times[url] = time.perf_counter() - start
        except requests.RequestException:
            times[url] = None
        time.sleep(1)
    return times

# Exercitiu 33
print(response_times(BASE_URL, BASE_URL + "/robots.txt"))
```

Rezultat: se afișează un dicționar cu două intrări, în care fiecare adresă are asociată durata cererii în secunde.



*Figura 3.13 - Exercițiul 33: funcția response_times(*urls)*

Exercițiul 34. Argumente cu nume colectate

Cerință: Scrieți log(message, **details), care afișează mesajul urmat de fiecare detaliu, în formatul din lucrare (de ex. verificat | url=... | status=200).

Funcția `log(message, **details)` colectează argumentele cu nume într-un dicționar `details`. Pentru fiecare pereche se construiește un fragment `cheie=valoare`, iar fragmentele (precedate de mesaj) sunt unite cu separatorul `|` .

```
def log(message: str, **details) -> None:
    """Afiseaza mesajul urmat de fiecare detaliu, in forma: mesaj | cheie=valoare | ..."""
    parts = [message] + [f"{key}={value}" for key, value in details.items()]
    print(" | ".join(parts))

# Exercitiu 34
log("verificat", url=BASE_URL, status=200)
```

Rezultat:

```
verificat | url=https://cybercor.org | status=200
```



*Figura 3.14 - Exercițiul 34: funcția log(message, **details)*

Întrebare de verificare

Întrebare: Care este diferența dintre a afișa o valoare cu print() în interiorul unei funcții și a o returna cu return?

`print()` doar **afișează** valoarea pe ecran: după ce funcția se termină, valoarea se pierde, iar programul nu o mai poate folosi. `return` **trimite valoarea înapoi** celui care a apelat funcția, care o poate salva într-o variabilă, o poate compara sau o poate da altei funcții (de exemplu `score_headers(security_headers(url))`). De aceea funcțiile din lucrare returnează valori, iar afișarea se face în afara lor, în codul care le apelează.

Partea 4. Module standard

Fișier de lucru: part4_modules.py (exercițiile 35–43). Fiecare soluție se scrie într-o funcție.

Exercițiul 35. Descompunerea unui URL

Cerință: Aplicați urllib.parse.urlparse() pe https://cybercor.org/path?x=1#top și afișați .scheme, .netloc, .path, .query și .fragment.

Funcția `urllib.parse.urlparse()` descompune un URL într-un obiect cu părțile sale: `.scheme` (protocolul), `.netloc` (domeniul), `.path` (calea), `.query` (parametrii) și `.fragment` (ancora de după `#`). A fost aplicată pe `https://cybercor.org/path?x=1#top`.

```
# Exercitiu 35
parsed = urlparse("https://cybercor.org/path?x=1#top")
print(parsed.scheme)
print(parsed.netloc)
print(parsed.path)
print(parsed.query)
print(parsed.fragment)
```

Rezultat:

```
https
cybercor.org
/path
x=1
top
```

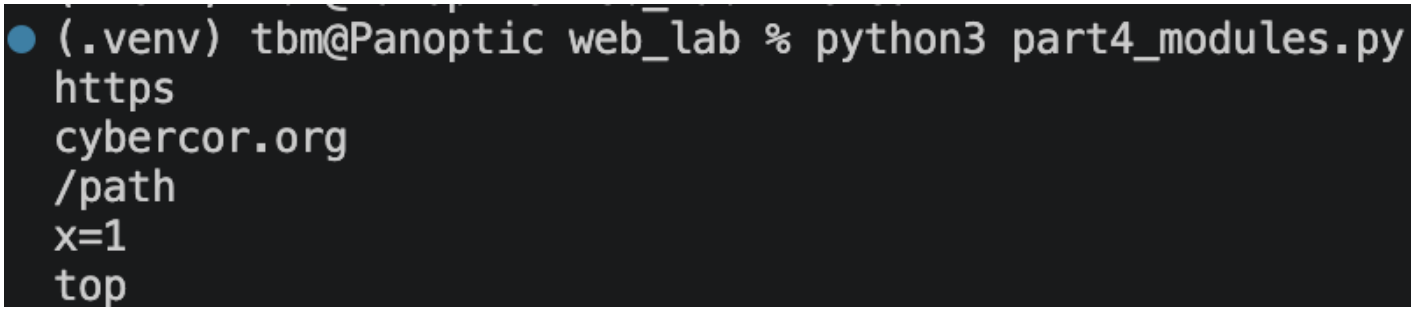


Figura 4.1 - Exercițiul 35: descompunerea unui URL cu urlparse()

Exercițiul 36. Compunerea URL-urilor

Cerință: Folosiți urllib.parse.urljoin() pentru a transforma legăturile relative /about, contact.html și ../index.html în URL-uri complete, pornind de la BASE_URL.

Funcția `urllib.parse.urljoin(base, link)` transformă o legătură relativă într-un URL complet, pornind de la adresa de bază. Cele trei legături au fost completate cu `BASE_URL`; legătura `../index.html` nu poate urca deasupra rădăcinii site-ului, deci ajunge la `/index.html`.

```
# Exercitiu 36
for link in ["/about", "contact.html", "../index.html"]:
    print(link, "->", urljoin(BASE_URL, link))
```

Rezultat:

```
/about -> https://cybercor.org/about
contact.html -> https://cybercor.org/contact.html
../index.html -> https://cybercor.org/index.html
```

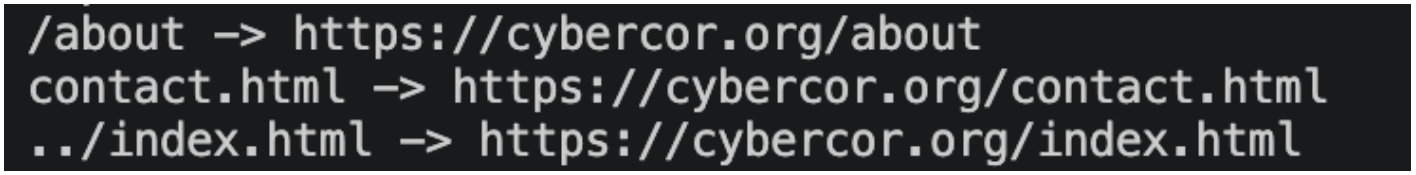


Figura 4.2 - Exercițiul 36: compunerea URL-urilor cu urljoin()

Exercițiul 37. Extragerea legăturilor

Cerință: Cu re.findall() obțineți legăturile (href) de pe pagina principală și puneți codul în extract_links(html), care returnează legăturile fără duplicate.

Expresia regulată `href="([^"]+)"` caută atributele `href` și returnează, prin grupul dintre paranteze, textul dintre ghilimele. Codul a fost pus în funcția `extract_links(html)`; `list(dict.fromkeys(links))` elimină duplicatele și păstrează ordinea din pagină (cheile unui dicționar sunt unice, iar ordinea de inserare se păstrează). Funcția a fost aplicată pe pagina principală. Limitare: expresia nu găsește legăturile scrise cu apostrof (`href='...'`).

```
# Exercitiu 37
def extract_links(html: str) -> list:
    """Returneaza legaturile (href) din codul HTML, fara duplicate, in ordinea din pagina."""
    links = re.findall(r'href="([^"]+)"', html)
    return list(dict.fromkeys(links))  # dict pastreaza ordinea si elimina duplicatele


page = fetch(BASE_URL)
links = extract_links(page.text)
print(len(links), "legaturi unice")
print(links)
```

Rezultat: se afișează numărul de legături unice și lista lor.

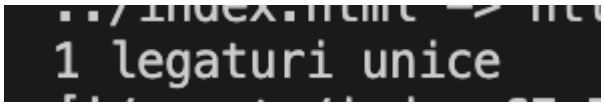


Figura 4.3 - Exercițiul 37: extragerea legăturilor cu extract_links(html)

Exercițiul 38. Interne sau externe?

Cerință: Scrieți split_links(links, domain), care returnează două liste: legăturile de pe același domeniu și legăturile către alte site-uri (urljoin(), apoi urlparse().netloc).

Funcția `split_links(links, domain)` transformă fiecare legătură într-un URL complet cu `urljoin()`, apoi compară `urlparse(url).netloc` cu domeniul dat: legăturile de pe același domeniu sunt interne, celelalte sunt externe. Funcția returnează două liste, `(interne, externe)`. Legăturile care nu sunt pagini web (`mailto:`, `tel:`, `javascript:`) sunt ignorate, pentru că schema lor nu este `http` sau `https`.

```
# Exercitiu 38
def split_links(links: list, domain: str) -> tuple:
    """Imparte legaturile in doua liste: (interne, externe) in raport cu domeniul dat.

    Legaturile care nu sunt pagini web (mailto:, tel:, javascript:) sunt ignorate.
    """
    base = "https://" + domain
    internal = []
    external = []
    for link in links:
        full_url = urljoin(base, link)
        parts = urlparse(full_url)
        if parts.scheme not in ("http", "https"):
            continue
        if parts.netloc == domain:
            internal.append(full_url)
        else:
            external.append(full_url)
    return internal, external


internal_links, external_links = split_links(links, "cybercor.org")
print(len(internal_links), "interne,", len(external_links), "externe")
```

Rezultat: se afișează numărul legăturilor interne și al celor externe ale paginii principale.



Figura 4.4 - Exercițiul 38: separarea legăturilor interne și externe

Exercițiul 39. Propriile metode

Cerință: Creați o clasă care moștenește html.parser.HTMLParser și suprascrie handle_starttag(), pentru a colecta atributul src al fiecărui tag <img>. Testați mai întâi pe fragmentul HTML din lucrare (assert), apoi pe site-ul real; verificați rezultatul prin comparație cu response.text.lower().count("<img") și cu codul sursă al paginii (Ctrl+U).

A fost creată clasa `ImageFinder`, care moștenește `html.parser.HTMLParser` și suprascrie metoda `handle_starttag()`. Pentru fiecare tag de deschidere, parserul apelează metoda cu numele tag-ului și lista atributelor; dacă tag-ul este `img` și are atributul `src`, valoarea este adăugată în lista `self.images`. `HTMLParser` transformă numele tag-urilor și ale atributelor în litere mici (deci `<IMG SRC=...>` este găsit), tag-urile care se închid singure (`<img ... />`) sunt tratate ca tag-uri de deschidere, tag-urile `<img>` fără `src` sunt ignorate prin `if src`, iar celelalte tag-uri (de exemplu `<a>`) prin condiția `tag == "img"`.

```
# Exercitiu 39
class ImageFinder(HTMLParser):
    """Parser HTML care colecteaza atributul src al fiecarui tag <img>."""

    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        if tag == "img":  # HTMLParser transforma numele tag-urilor in litere mici
            src = dict(attrs).get("src")
            if src:  # ignoram <img> fara src
                self.images.append(src)


test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)

assert finder.images == [
    "/logo.png",                            # imagine obisnuita
    "poza.jpg",                             # tag scris cu majuscule
    "https://cdn.example.com/banner.webp",  # tag care se inchide singur
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul a trecut!")
```

Testul pe HTML cunoscut: parserul a fost testat mai întâi pe fragmentul din lucrare, pentru care rezultatul corect se știe dinainte. Instrucțiunea `assert` a trecut, au fost găsite exact cele trei imagini așteptate, în ordinea din pagină, și s-a afișat „Testul a trecut!”:

```
['/logo.png', 'poza.jpg', 'https://cdn.example.com/banner.webp']
Testul a trecut!
```

Rularea pe site-ul real: abia după test, parserul a fost rulat pe pagina `BASE_URL`.

```
response = requests.get(BASE_URL, timeout=TIMEOUT)
finder = ImageFinder()
finder.feed(response.text)
print(len(finder.images), "imagini găsite")
for src in finder.images:
    print(src)
print("Tag-uri <img> in text:", response.text.lower().count("<img"))
# Daca cele doua numere difera, uitati-va in sursa paginii (Ctrl+U): unele <img> pot
# lipsi de src (de ex. imagini incarcate cu JavaScript, atribut data-src)
```

Rezultat: se afișează numărul de imagini găsite și sursa fiecăreia, apoi numărul de apariții ale textului `<img` în pagină. Rezultatul a fost verificat în două moduri: prin compararea celor două numere și prin deschiderea paginii în browser și vizualizarea codului sursă (Ctrl+U). Dacă numerele diferă, cauza poate fi un tag `<img>` fără `src` (de exemplu imagini încărcate cu JavaScript, prin atributul `data-src`); o listă goală nu înseamnă neapărat o greșeală, deoarece unele site-uri afișează imaginile prin CSS sau SVG, nu prin tag-uri `<img>`.

```
['/logo.png', 'poza.jpg', 'https://cdn.example.com/banner.webp']
Testul a trecut!
```

Figura 4.5 - Exercițiul 39: clasa ImageFinder și testul pe fragmentul HTML cunoscut

```
0 imagini găsite
Tag-uri <img> in text: 0
```

Figura 4.6 - Exercițiul 39: ImageFinder pe site-ul real și compararea cu numărul de tag-uri <img>

Exercițiul 40. Amprenta paginii

Cerință: Scrieți page_fingerprint(url), care returnează hash-ul SHA-256 al conținutului; rulați-o de două ori, comparați rezultatele și explicați când ar fi diferite.

Funcția `page_fingerprint(url)` descarcă pagina și returnează amprenta ei: `hashlib.sha256(response.content).hexdigest()`, un șir hexazecimal de 64 de caractere. Funcția a fost rulată de două ori (cu pauză de o secundă), iar rezultatele au fost comparate.

```
# Exercitiu 40
def page_fingerprint(url: str) -> str:
    """Returneaza amprenta SHA-256 (in hexazecimal) a continutului paginii."""
    response = fetch(url)
    return hashlib.sha256(response.content).hexdigest()
```

```
first = page_fingerprint(BASE_URL)
time.sleep(1)
second = page_fingerprint(BASE_URL)
print(first)
print(second)
print("Identice:", first == second)
# Amprentele ar fi diferite daca pagina se schimba intre cele doua cereri: continut
# dinamic (data si ora, contor de vizitatori, reclame, token-uri sau nonce-uri
# generate la fiecare cerere), o actualizare a site-ului sau servere diferite
# (balansare de incarcare) care trimit versiuni diferite
```

Rezultat: se afișează cele două amprente și valoarea `Identice: True` sau `Identice: False`. Pentru o pagină statică amprentele sunt identice; orice diferență de un singur octet produce o amprentă complet diferită.

Amprentele ar fi diferite dacă pagina se schimbă între cele două cereri: conținut dinamic (data și ora, contor de vizitatori, reclame, token-uri sau valori `nonce` generate la fiecare cerere), o actualizare a site-ului sau servere diferite (balansarea încărcării) care trimit versiuni diferite ale paginii.

```
7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292
7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292
```

Figura 4.7 - Exercițiul 40: amprenta paginii (SHA-256), rulată de două ori

Exercițiul 41. Salvarea în JSON

Cerință: Salvați antetele paginii principale în headers.json (json.dump, indent=2); apoi încărcați fișierul înapoi cu json.load() și afișați un antet din el.

Funcția `save_headers(url, filename)` salvează antetele paginii în fișierul `headers.json` cu `json.dump(dict(response.headers), file, indent=2)`. Conversia `dict(...)` este necesară, deoarece `response.headers` este un `CaseInsensitiveDict`, pe care `json` nu îl poate serializa direct. Apoi fișierul este încărcat înapoi cu `json.load()`, iar primul antet din el este afișat.

```
# Exercitiu 41
def save_headers(url: str, filename: str = "headers.json") -> dict:
    """Salveaza antetele paginii in fisierul JSON filename si le incarca inapoi."""
    response = fetch(url)
    with open(filename, "w", encoding="utf-8") as file:
        json.dump(dict(response.headers), file, indent=2)
    with open(filename, "r", encoding="utf-8") as file:
        return json.load(file)


time.sleep(1)
loaded = save_headers(BASE_URL)
first_name = next(iter(loaded))
print(first_name + ":", loaded[first_name])
```

Rezultat: se creează fișierul `headers.json` (indentat cu 2 spații) și se afișează un antet citit din el, sub forma `Nume: valoare`.



Figura 4.8 - Exercițiul 41: salvarea și încărcarea antetelor (headers.json)

Exercițiul 42. Interogare DNS

Cerință: Scrieți resolve(hostname), care returnează adresa IP (socket.gethostbyname()); afișați adresa IP a domeniului cybercor.org.

Funcția `resolve(hostname)` returnează adresa IP a domeniului cu `socket.gethostbyname()` (interogare DNS). Dacă domeniul nu poate fi rezolvat, eroarea `socket.gaierror` este prinsă și funcția returnează `None`.

```
# Exercitiu 42
def resolve(hostname: str) -> Optional[str]:
    """Returneaza adresa IP a domeniului hostname, sau None daca domeniul nu se rezolva."""
    try:
        return socket.gethostbyname(hostname)
    except socket.gaierror:
        return None


print("Adresa IP:", resolve("cybercor.org"))
```

Rezultat: se afișează adresa IP (IPv4) a domeniului `cybercor.org`.



Figura 4.9 - Exercițiul 42: interogarea DNS (adresa IP a domeniului)

Exercițiul 43. Expirarea certificatului

Cerință: Scrieți cert_days_left(hostname), care se conectează la portul 443, citește certificatul și returnează numărul de zile rămase până la expirare.

Funcția `cert_days_left(hostname)` se conectează la portul 443 și citește certificatul serverului, în etapele: (1) `ssl.create_default_context()` creează un context cu verificarea certificatului activată; (2) `socket.create_connection((hostname, 443), timeout=TIMEOUT)` deschide conexiunea TCP; (3) `context.wrap_socket(sock, server_hostname=hostname)` stabilește conexiunea TLS; (4) `getpeercert()` returnează certificatul ca dicționar, din care se ia câmpul `notAfter` (data expirării); (5) `ssl.cert_time_to_seconds()` convertește data în secunde, iar diferența față de momentul curent, în zile (`.days`), este valoarea returnată.

```
# Exercitiu 43
def cert_days_left(hostname: str) -> int:
    """Returneaza cate zile mai sunt pana la expirarea certificatului SSL al domeniului."""
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443), timeout=TIMEOUT) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as secure_sock:
            cert = secure_sock.getpeercert()
    expires_at = datetime.fromtimestamp(
        ssl.cert_time_to_seconds(cert["notAfter"]), tz=timezone.utc
    )
    return (expires_at - datetime.now(timezone.utc)).days


print("Zile pana la expirarea certificatului:", cert_days_left("cybercor.org"))
```

Rezultat: se afișează numărul de zile rămase până la expirarea certificatului SSL al domeniului `cybercor.org`.

```
Zile pana la expirarea certificatului: 48
(.venv) tbm@Panoptic web_lab %
```

Figura 4.10 - Exercițiul 43: zilele rămase până la expirarea certificatului

Întrebare de verificare

Întrebare: În exercițiul 39, cine apelează handle_starttag()? (voi nu o apelați niciodată direct)

Metoda `handle_starttag()` este apelată de **parserul însuși**. Programul apelează doar `finder.feed(html)`; în interiorul lui `HTMLParser`, textul este parcurs, iar de fiecare dată când este recunoscut un tag de deschidere, parserul apelează `handle_starttag(tag, attrs)` (prin `feed()` → `goahead()` → `parse_starttag()` → `handle_starttag()`). Metoda din clasa de bază nu face nimic, iar noi o suprascriem pentru a spune ce să se întâmple la fiecare tag. Este principiul „nu ne apela, te apelăm noi”: codul nostru este apelat de bibliotecă (callback).

Partea 5. Modulul propriu

Fișier de lucru: webtools.py, main.py (exercițiile 44–50).

Exercițiul 44. Crearea unui modul

Cerință: Mutați funcțiile fetch, get_status, get_title și security_headers în webtools.py; în main.py importați webtools și apelați webtools.get_title(webtools.fetch(BASE_URL).text).

Funcțiile `fetch`, `get_status`, `get_title` și `security_headers` au fost mutate în fișierul `webtools.py`, care devine un modul propriu, importabil ca orice altă bibliotecă. În `webtools.py` au fost adăugate și celelalte funcții necesare raportului final (`score_headers`, `check_paths`, `fetch_robots`, `disallowed_paths`, `extract_links`, `split_links`, `resolve`, `cert_days_left`, `redirect_chain`, `save_csv`, `site_report`), astfel încât `main.py` să rămână scurt. În `main.py` modulul este importat cu `import webtools`, iar funcțiile se apelează cu prefixul `webtools.` (adresa vine din linia de comandă, vezi exercițiul 48).

```
# webtools.py
def fetch(url: str, timeout: int = TIMEOUT) -> requests.Response:
    """Descarca adresa url (cu antetele din DEFAULT_HEADERS) si returneaza raspunsul."""
    return requests.get(url, headers=DEFAULT_HEADERS, timeout=timeout)

def get_status(url: str) -> int:
    """Returneaza codul de stare HTTP pentru adresa url."""
    return fetch(url).status_code

def get_title(html: str) -> Optional[str]:
    """Returneaza titlul paginii (textul dintre <title> si </title>), sau None."""
    lower_html = html.lower()
    tag_start = lower_html.find("<title>")
    if tag_start == -1:
        return None
    start = tag_start + len("<title>")
    end = lower_html.find("</title>")
    return html[start:end].strip()

def security_headers(url: str) -> dict:
    """Verifica cele cinci antete de securitate si returneaza {antet: True/False}."""
    headers = fetch(url).headers
    return {name: name in headers for name in SECURITY_HEADERS}

# main.py
import webtools  # Exercitiu 44: importam tot modulul si scriem webtools.nume

# Exercitiu 44
page = webtools.fetch(url)
print("Titlu (webtools.get_title):", webtools.get_title(page.text))
```

Rezultat: se afișează linia `Titlu (webtools.get_title): ...` cu titlul paginii.

```
(.venv) tbm@Panoptic web_lab % head webtools.py
# Laborator: functii, metode si importuri pe web
# Student: Berejnoi Alexandr
# Modulul meu: toate functiile reutilizabile

import csv
import json
import re
import socket
import ssl
import time
(.venv) tbm@Panoptic web_lab % head main.py
# Laborator: functii, metode si importuri pe web
# Student: Berejnoi Alexandr

import argparse
import time

import webtools  # Exercitiu 44: importam tot modulul si scriem webtools.nume
from webtools import get_title, security_headers  # Exercitiu 45: importam doar numele dorite
from webtools import DEFAULT_HEADERS  # Exercitiu 47: importam constanta

(.venv) tbm@Panoptic web_lab % python3 main.py
usage: main.py [-h] url
main.py: error: the following arguments are required: url
(.venv) tbm@Panoptic web_lab %
```

Figura 5.1 - Exercițiul 44: modulul webtools și apelul din main.py

Exercițiul 45. Importarea anumitor nume

Cerință: În main.py treceți la from webtools import get_title, security_headers; explicați într-un comentariu ce s-ar întâmpla dacă main.py ar avea și o funcție proprie get_title.

În `main.py` s-a trecut la importul numelor dorite, `from webtools import get_title, security_headers`; funcțiile se folosesc acum direct, fără prefixul `webtools.`.

```
from webtools import get_title, security_headers  # Exercitiu 45: importam doar numele dorite

# Exercitiu 45
# Daca main.py ar avea si o functie proprie numita get_title, ea ar inlocui functia
# importata: ultima definitie a unui nume ramane valabila si Python nu da nicio eroare.
# Apelurile get_title(...) ar rula functia din main.py, iar functia din webtools ar putea
# fi folosita doar ca webtools.get_title(...). Din acest motiv, importul "from ... import"
# poate crea confuzii cand numele se repeta

print("Titlu (get_title importat):", get_title(page.text))
print("Antete de securitate:", security_headers(url))
```

Rezultat: se afișează titlul paginii (apelat fără prefix) și dicționarul antetelor de securitate.

Dacă `main.py` ar avea și o funcție proprie numită `get_title`, ea ar **înlocui** funcția importată: ultima definiție a unui nume rămâne valabilă, iar Python nu dă nicio eroare. Apelurile `get_title(...)` ar rula funcția din `main.py`, iar funcția din `webtools` ar putea fi

folosită doar ca `webtools.get_title(...)`. Din acest motiv, importul de tip `from ... import` poate crea confuzii când numele se repetă.

```
from webtools import get_title, security_headers  # Exercitiu 45: importam doar numele dorite
```

Figura 5.2 - Exercițiul 45: importul numelor get_title și security_headers

Exercițiul 46. Blocul main

Cerință: Adăugați la finalul webtools.py blocul if __name__ == "__main__" cu autotestul din lucrare; rulați python webtools.py, apoi python main.py și explicați de ce autotestul rulează doar în primul caz.

La finalul fișierului `webtools.py` a fost adăugat blocul `if __name__ == "__main__":` cu autotestul din lucrare. Fișierul a fost rulat direct (`python webtools.py`), apoi indirect, prin `python main.py https://cybercor.org`, care îl importă.

```
# Exercitiu 46
# Blocul de mai jos ruleaza doar cand fisierul este pornit direct (python webtools.py).
# In acest caz Python pune in variabila __name__ valoarea "__main__". Cand fisierul este
# importat (de ex. din main.py), __name__ este "webtools", conditia este falsa si
# autotestul nu mai ruleaza, deci importul nu face cereri web nedorite
if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Rezultat: la `python webtools.py` se afișează linia `Autotest: <cod de stare>` (200 pentru o cerere reușită). La `python main.py ...` linia `Autotest:` nu apare.

Când un fișier este pornit direct, Python îi dă variabilei `__name__` valoarea `"__main__"`, deci condiția este adevărată și autotestul rulează. Când fișierul este importat (de exemplu din `main.py`), `__name__` este numele modulului, `"webtools"`, condiția este falsă și autotestul nu mai rulează; astfel importul nu face cereri web nedorite.

```
(.venv) tbm@Panoptic web_lab % python3 webtools.py
Autotest: 200
```

Figura 5.3 - Exercițiul 46: rularea python webtools.py (autotestul rulează)

```
(.venv) tbm@Panoptic web_lab % python3 main.py
usage: main.py [-h] url
main.py: error: the following arguments are required: url
```

Figura 5.4 - Exercițiul 46: rularea python main.py (autotestul nu rulează)

Exercițiul 47. Constante de modul

Cerință: Adăugați în webtools.py constanta DEFAULT_HEADERS (User-Agent WebLab-<numele vostru>) și folosiți-o în fetch(); în main.py importați constanta și afișați-o.

În `webtools.py` a fost adăugată constanta de modul `DEFAULT_HEADERS` cu antetul `User-Agent`, care este transmisă în `fetch()` prin `headers=DEFAULT_HEADERS`; astfel toate cererile făcute prin `fetch()` poartă același `User-Agent`. În `main.py` constanta este importată cu `from webtools import DEFAULT_HEADERS` și afișată.

```
# Exercitiu 47
DEFAULT_HEADERS = {"User-Agent": "WebLab-Berejnoi-Alexandr"}

def fetch(url: str, timeout: int = TIMEOUT) -> requests.Response:
    """Descarca adresa url (cu antetele din DEFAULT_HEADERS) si returneaza raspunsul."""
    return requests.get(url, headers=DEFAULT_HEADERS, timeout=timeout)

# Exercitiu 47
print("DEFAULT_HEADERS:", DEFAULT_HEADERS)
```

Rezultat:

```
DEFAULT_HEADERS: {'User-Agent': 'WebLab-Berejnoi-Alexandr'}
```

```
SECURITY_HEADERS = [
    "Strict-Transport-Security",
    "Content-Security-Policy",
    "X-Frame-Options",
    "X-Content-Type-Options",
    "Referrer-Policy",
]
```

Figura 5.5 - Exercițiul 47: constanta DEFAULT_HEADERS în webtools.py și main.py

Exercițiul 48. Argumente din linia de comandă

Cerință: Folosiți argparse în main.py, astfel încât programul să ruleze ca python main.py https://cybercor.org; transmiteți URL-ul primit funcțiilor în loc de BASE_URL.

În `main.py` a fost folosit modulul `argparse`: `ArgumentParser` definește argumentul pozițional `url`, iar `parse_args()` îl citește din linia de comandă. Programul se rulează

`python main.py https://cybercor.org`, iar URL-ul primit este transmis tuturor funcțiilor, în locul lui `BASE_URL`. Adresa este verificată: dacă nu începe cu `http://` sau `https://`, `parser.error()` afișează un mesaj de eroare și oprește programul. Dacă argumentul lipsește, `argparse` afișează automat mesajul de utilizare (`usage: main.py [-h] url`) și eroarea „the following arguments are required: url”.

```
# Exercitiu 48
parser = argparse.ArgumentParser(description="Raport despre un site web")
parser.add_argument("url", help="adresa site-ului, de ex. https://cybercor.org")
args = parser.parse_args()
url = args.url
if not url.startswith(("http://", "https://")):
    parser.error("adresa trebuie sa inceapa cu http:// sau https://")
```

Rezultat: programul rulează pe adresa primită ca argument.

```
(.venv) tbm@Panoptic web_lab % python3 main.py example.com
usage: main.py [-h] url
main.py: error: adresa trebuie sa inceapa cu http:// sau https://
(.venv) tbm@Panoptic web_lab % python3 main.py https://example.com
```

Figura 5.6 - Exercițiul 48: argumentul din linia de comandă (argparse)

Exercițiul 49. Un raport CSV

Cerință: Cu modulul csv salvați rezultatul funcției check_paths() în report.csv, cu coloanele path, status și checked_at (ora: datetime.now().isoformat()).

Funcția `save_csv(results, filename)` scrie rezultatul funcției `check_paths()` în fișierul `report.csv`, cu `csv.writer`. Prima linie conține coloanele `path`, `status`, `checked_at`, apoi urmează câte un rând pentru fiecare cale: calea, codul de stare și ora verificării, `datetime.now().isoformat()`. Fișierul este deschis cu `newline=""`, așa cum recomandă documentația modulului `csv`, pentru a evita liniile goale suplimentare. În `main.py`, `check_paths` verifică căile `/`, `/robots.txt` și `/sitemap.xml`, iar rezultatul este salvat.

```
def save_csv(results: dict, filename: str = "report.csv") -> None:
    """Salveaza rezultatul check_paths in fisierul CSV, cu coloanele path, status,
checked_at."""
    with open(filename, "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow(["path", "status", "checked_at"])
        for path, status in results.items():
            writer.writerow([path, status, datetime.now().isoformat()])

# Exercitiu 49
results = webtools.check_paths(url, ["/", "/robots.txt", "/sitemap.xml"])
webtools.save_csv(results, "report.csv")
```

```
print("Salvat in report.csv")
print()
```

Rezultat: se creează fișierul `report.csv` cu un rând de antet și trei rânduri de date; ora este în format ISO (de exemplu `2026-10-14T10:15:32.123456`).

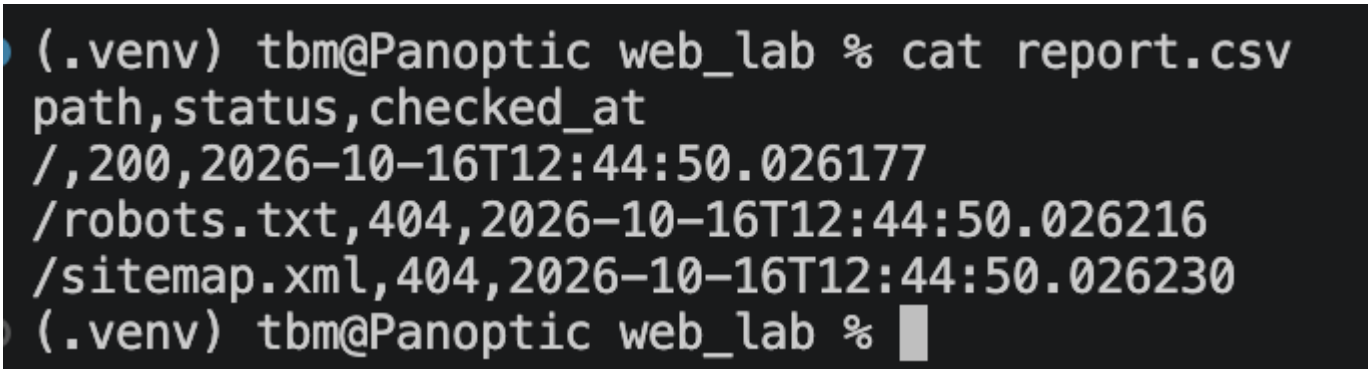


Figura 5.7 - Exercițiul 49: fișierul report.csv

Exercițiul 50. Proiect final: raportul site-ului

Cerință: Scrieți site_report(url) în webtools.py; funcția folosește funcțiile voastre pentru a colecta, a afișa și a salva în report.json: (1) codul de stare și URL-ul final, (2) titlul, (3) adresa IP, (4) lanțul de redirecționări pornind de la http://, (5) scorul antetelor de securitate, (6) zilele rămase până la expirarea certificatului, (7) numărul de legături interne și externe, (8) căile interzise din robots.txt.

Funcția `site_report(url)` din `webtools.py` colectează informațiile cerute folosind funcțiile scrise în exercițiile anterioare, le afișează și le salvează în `report.json`:

1) codul de stare și URL-ul final: `fetch()`, `response.status_code`, `response.url`; 2) titlul: `get_title()`; 3) adresa IP: `resolve()`; 4) lanțul de redirecționări pornind de la `http://`: `redirect_chain()` (funcție nouă, care cere varianta `http://` a site-ului și construiește șiruri de forma `301 -> https://cybercor.org/` din `response.history`); 5) scorul antetelor de securitate: `score_headers(security_headers(url))`; 6) zilele rămase până la expirarea certificatului: `cert_days_left()`; 7) numărul de legături interne și externe: `extract_links()` și `split_links()`; 8) căile interzise din `robots.txt`: `disallowed_paths(fetch_robots(url))`.

Funcția este robustă: dacă domeniul nu se rezolvă sau certificatul nu poate fi citit (`OSError`, care include `ssl.SSLError`), valorile devin `None`, iar raportul afișează „necunoscută” sau „indisponibil” în loc să se oprească. Între cereri există `time.sleep(1)`, iar datele sunt salvate cu `json.dump(..., indent=2, ensure_ascii=False)`.

```
def redirect_chain(url: str) -> list:
    """Porneste de la varianta http:// a site-ului si returneaza lista redirectionarilor.

    Fiecare element are forma '301 -> https://exemplu.ro/'. Lista este goala daca
    nu exista redirectionari.
    """
    host = urlparse(url).netloc
    response = fetch("http://" + host)
    steps = response.history + [response]
    return [f"{now.status_code} -> {after.url}" for now, after in zip(steps, steps[1:])]

def site_report(url: str, filename: str = "report.json") -> dict:
    """Aduna informatii despre site, le afiseaza si le salveaza in report.json.

    Foloseste functiile din acest modul. Intre cereri se face o pauza de o secunda.
    Returneaza dictionarul cu informatiile colectate.
    """
    host = urlparse(url).netloc

    response = fetch(url)
    status = response.status_code
    final_url = response.url
    title = get_title(response.text)
    links = extract_links(response.text)
    internal, external = split_links(links, host)
    time.sleep(1)

    ip = resolve(host)

    redirects = redirect_chain(url)
    time.sleep(1)

    score = score_headers(security_headers(url))
    time.sleep(1)

    try:
        days_left = cert_days_left(host)
    except OSError:  # include si ssl.SSLError si erorile de conectare
        days_left = None

    disallowed = disallowed_paths(fetch_robots(url))

    report = {
        "url": url,
        "status": status,
        "final_url": final_url,
        "title": title,
        "ip": ip,
        "redirects": redirects,
        "security_score": score,
        "cert_days_left": days_left,
        "internal_links": len(internal),
        "external_links": len(external),
        "disallowed": disallowed,
    }

    def line(label: str, value) -> None:
        print(f"{label:<19}{value}")

    print(f"=== Raport site: {url} ===")
    line("Cod de stare:", status)
    line("URL final:", final_url)
    line("Titlu:", title if title is not None else "lipsește")
    line("Adresă IP:", ip if ip is not None else "necunoscută")
    line("Redirecționări:", ", ".join(redirects) if redirects else "fără redirecționări")
```

```
    line("Scor securitate:", score)
    if days_left is None:
        line("Certificat:", "indisponibil")
    else:
        line("Certificat:", f"{days_left} de zile rămase")
    line("Legături:", f"{len(internal)} interne, {len(external)} externe")
    line("Căi interzise:", ", ".join(disallowed) if disallowed else "niciuna")

    with open(filename, "w", encoding="utf-8") as file:
        json.dump(report, file, indent=2, ensure_ascii=False)
    print(f"Salvat în {filename}")
    return report
```

Rezultat: programul afișează raportul în formatul din lucrare (`=== Raport site: ... ===`, urmat de liniile: Cod de stare, URL final, Titlu, Adresă IP, Redirecționări, Scor securitate, Certificat, Legături și Căi interzise) și mesajul „Salvat în report.json”. Fișierul `report.json` conține cheile `url`, `status`, `final_url`, `title`, `ip`, `redirects`, `security_score`, `cert_days_left`, `internal_links`, `external_links` și `disallowed`.

```
(.venv) tbm@Panoptic web_lab % python3 main.py https://cybercor.org
Titlu (webtools.get_title): CYBERCOR
Titlu (get_title importat): CYBERCOR
Antete de securitate: {'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': T
'Referrer-Policy': True}
DEFAULT_HEADERS: {'User-Agent': 'WebLab-X-Y'}
Salvat in report.csv

=== Raport site: https://cybercor.org ===
Cod de stare:      200
URL final:         https://cybercor.org/
Titlu:             CYBERCOR
Adresă IP:         172.67.211.96
Redirecționări:    301 -> https://cybercor.org/
Scor securitate:   3/5
Certificat:        48 de zile rămase
Legături:          1 interne, 0 externe
Căi interzise:     niciuna
Salvat în report.json
(.venv) tbm@Panoptic web_lab %
```

Figura 5.8 - Exercițiul 50: raportul site-ului afișat de python main.py https://cybercor.org

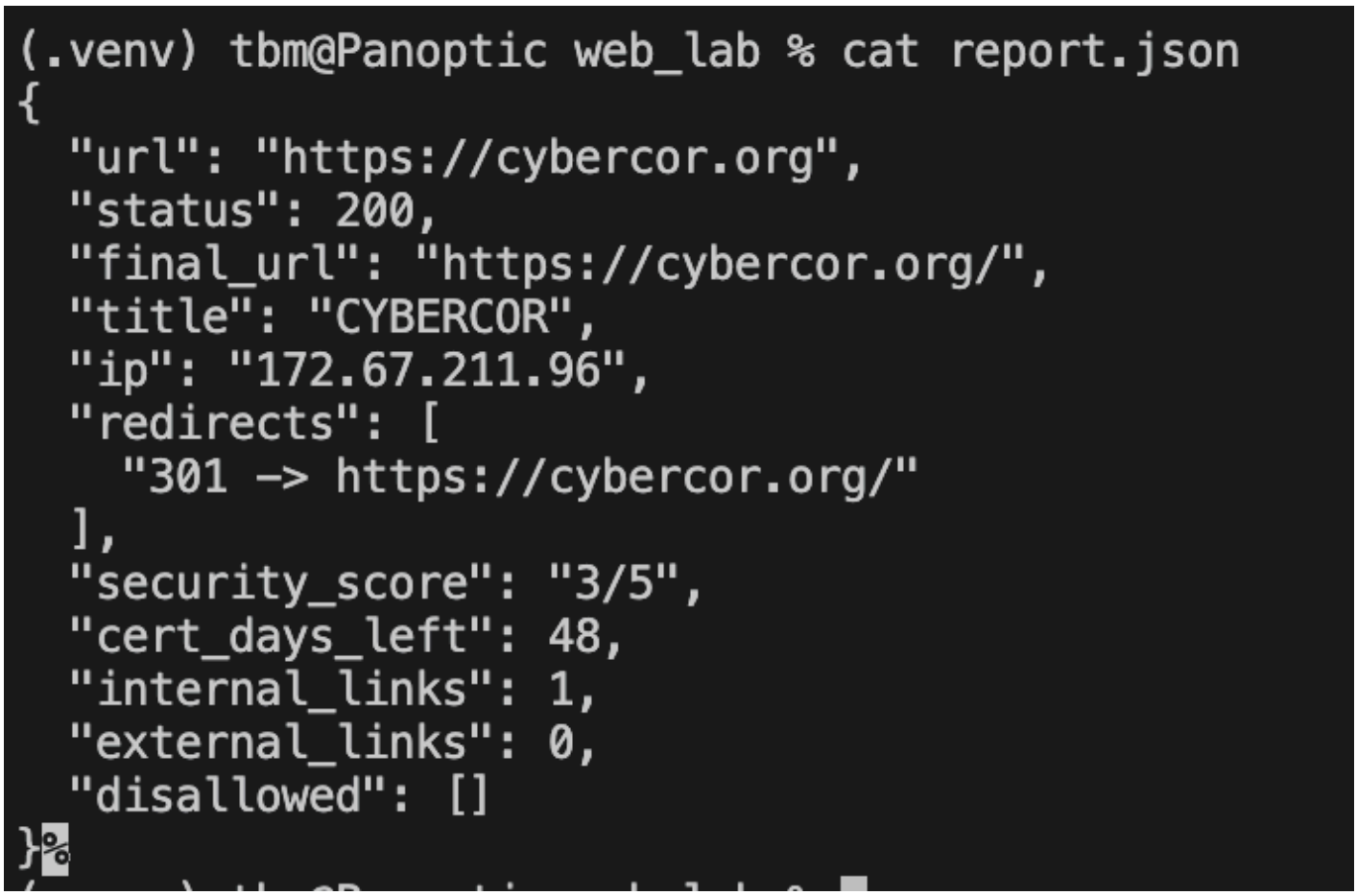


Figura 5.9 - Exercițiul 50: conținutul fișierului report.json

Structura finală a directorului web_lab

Cerință: Directorul trebuie să conțină webtools.py, main.py, report.csv și report.json, iar comanda python main.py https://cybercor.org trebuie să afișeze raportul complet și să creeze report.json.

La finalul lucrării au fost efectuate următoarele verificări:

• directorul `web_lab` conține fișierele `webtools.py` (modulul propriu), `main.py` (programul care îl importă), `report.csv` (creat în exercițiul 49) și `report.json` (creat în exercițiul 50), precum și fișierele de lucru `part1_imports.py`–`part4_modules.py` și `headers.json` (exercițiul 41);

• comanda `python main.py https://cybercor.org` rulează fără erori, afișează raportul complet al site-ului și creează `report.json` (precum și `report.csv`);

• fiecare cerere web folosește `timeout`, între cererile repetate există `time.sleep(1)`, către `cybercor.org` se trimit doar cereri GET și HEAD, iar în cod nu există bucle `while True` care trimit cereri;

• funcțiile definite în părțile 3–5 au docstring, iar toate cerințele de tip „explicați într-un comentariu” au răspuns în cod;

• fiecare fișier `partN_*.py` a fost rulat de la început până la sfârșit, fără erori.

```
web_lab/
├── webtools.py     # modulul propriu: toate funcțiile reutilizabile
├── main.py         # programul care importă webtools
├── report.csv      # creat în exercițiul 49
├── report.json     # creat în exercițiul 50
├── headers.json    # creat în exercițiul 41
└── part1_imports.py, part2_methods.py, part3_functions.py, part4_modules.py
```

```
(.venv) tbm@Panoptic web_lab % ls -la
total 96
drwxr-xr-x@ 13 tbm  staff   416 Oct 10 13:53 .
drwxr-xr-x  37 tbm  staff  1184 Oct 11 20:53 ..
drwxr-xr-x@  7 tbm  staff   224 Oct  3 17:14 .venv
drwxr-xr-x@  3 tbm  staff    96 Oct 11 20:47 __pycache__
-rw-r--r--@  1 tbm  staff   949 Oct 16 12:33 headers.json
-rw-r--r--@  1 tbm  staff  1877 Oct 11 14:02 main.py
-rw-r--r--@  1 tbm  staff  2857 Oct 16 11:42 part1_imports.py
-rw-r--r--@  1 tbm  staff  3522 Oct 11 14:42 part2_methods.py
-rw-r--r--@  1 tbm  staff  6431 Oct 11 14:02 part3_functions.py
-rw-r--r--@  1 tbm  staff  5699 Oct 11 20:44 part4_modules.py
-rw-r--r--@  1 tbm  staff   147 Oct 16 12:45 report.csv
-rw-r--r--@  1 tbm  staff   314 Oct 16 12:45 report.json
-rw-r--r--@  1 tbm  staff  8177 Oct 11 14:02 webtools.py
```

Figura 5.10 - Structura directorului web_lab și rularea finală a programului

CONCLUZII

În cadrul lucrării practice nr. 5 am realizat toate cele 50 de exerciții, grupate în cinci părți, iar rezultatul final este modulul propriu `webtools.py`, programul `main.py` și raportul unui site web în formatele CSV și JSON.

În prima parte am studiat importurile: diferența dintre un modul extern (`requests`, instalat cu `pip`) și unul din biblioteca standard (`urllib`), stilurile `import` și `from ... import`, alias-urile, citirea documentației cu `dir()` și `help()`, măsurarea timpului și tratarea

lipsei unui modul cu `ImportError`. În partea a doua au fost folosite atributele și metodele obiectelor web: răspunsul HTTP (`status_code`, `ok`, `url`, `history`), antetele (dicționar insensibil la majuscule), metodele de șir înlănțuite, diferența dintre HEAD și GET, cookie-urile și sesiunile.

În partea a treia au fost scrise funcții proprii, cu parametri impliciți, argumente cu nume, `*args` și `**kwargs`, docstring-uri, adnotări de tip și tratarea erorilor cu `try / except`, urmărind principiul „o funcție, o sarcină” și diferența dintre `print()` și `return`. În partea a patra au fost combinate module standard: `urllib.parse`, `re`, `html.parser` (clasă proprie, care suprascrie `handle_starttag()`), `hashlib`, `json`, `socket` și `ssl`. În partea a cincea funcțiile au fost organizate într-un modul propriu, cu constante de modul, blocul `if __name__ == "__main__"`, argumente din linia de comandă (`argparse`) și salvarea rezultatelor în `report.csv` și `report.json`.

Regulile de lucru cu un site real (timeout la fiecare cerere, pauze între cereri, doar GET și HEAD, fără bucle infinite) au fost respectate în toate fișierele. Rezultatul final, `python main.py https://cybercor.org`, afișează un raport complet despre site: codul de stare, titlul, adresa IP, redirecționările, scorul antetelor de securitate, valabilitatea certificatului, legăturile interne și externe și căile interzise din `robots.txt`.